

Full Stack Development with MERN

1.Introduction

Project Title

ShopEZ – MERN Stack Online Shopping Platform

Developer

Name	Role	Responsibilities
Akshara Cheedalla	Full Stack MERN Developer	Designed and developed the complete ShopEZ application using the MERN Stack. Developed the React.js frontend, implemented RESTful APIs using Node.js and Express.js, designed the MongoDB database, integrated JWT authentication, managed product, cart, and order modules, developed the admin dashboard, performed testing, debugging, deployment, and prepared the complete project documentation.

2. Project Overview

Purpose

ShopEZ is a Full Stack MERN-based Online Shopping Platform designed to provide customers with a secure, reliable, and user-friendly shopping experience. The platform enables users to browse products, search items, manage shopping carts, place orders, and track purchases through an intuitive web interface. Administrators can efficiently manage products, categories, users, banners, and orders through a dedicated dashboard.

Traditional online shopping platforms often suffer from poor user experience, complicated checkout procedures, fake reviews, payment security concerns, and inefficient order management. ShopEZ addresses these challenges by providing a modern, scalable, and secure e-commerce solution using React.js, Node.js, Express.js, and MongoDB.

Objectives

- Build a complete MERN Stack application.
- Provide secure user authentication using JWT.
- Allow users to browse and search products.
- Enable shopping cart management.
- Support secure checkout and order placement.
- Provide an admin dashboard for managing products, users, and orders.
- Design a scalable MVC-based backend architecture.

Features

Customer Features

- User Registration
- Secure Login
- JWT Authentication
- Product Listing
- Product Search
- Product Filtering
- Product Details
- Shopping Cart
- Checkout
- Order Confirmation
- Order Tracking
- Order History
- Profile Management

Administrator Features

- Admin Login
- Dashboard
- Product Management

- Category Management
 - User Management
 - Order Management
 - Banner Management
 - Sales Analytics
-

3. Architecture

Frontend Architecture

The frontend of ShopEZ is developed using React.js and follows a component-based architecture. Each feature of the application is implemented as reusable React components, enabling modularity, maintainability, and scalability.

Major frontend modules include:

- Home Page
- Login
- Registration
- Product Listing
- Product Details
- Search & Filter
- Shopping Cart
- Checkout
- Orders
- User Profile
- Admin Dashboard

React Router DOM is used for navigation between pages. Axios is used to communicate with backend APIs.

Backend Architecture

The backend is developed using Node.js and Express.js following the MVC (Model–View–Controller) architecture.

Controllers

- Auth Controller
- Product Controller
- Cart Controller
- Order Controller
- User Controller
- Admin Controller

Models

- User
- Product
- Cart
- Order
- Category

Routes

REST APIs handle communication between frontend and database.

Database Architecture

MongoDB is used as the NoSQL database.

Collections include:

- Users
- Products
- Categories
- Cart
- Orders
- Reviews

Relationships are maintained using MongoDB ObjectIds.

The backend communicates with MongoDB using Mongoose ODM.

4. Setup Instructions

Prerequisites

Install the following software before running the project.

- Node.js (v18 or above)
- npm
- MongoDB Community Server or MongoDB Atlas
- Git
- Visual Studio Code

Installation

Clone Repository

```
git clone https://github.com/username/Shopez.git
```

Install Frontend

```
cd client
```

```
npm install
```

Install Backend

```
cd server
```

```
npm install
```

Environment Variables

Create a **.env** file.

```
PORT=5000
```

```
MONGO_URI=your_mongodb_connection
```

```
JWT_SECRET=your_secret_key
```

Prerequisites

- Node.js v18+
- MongoDB (local `mongodb://localhost:27017` or Atlas URI)

1. Clone & Setup Server

```
cd server
npm install
# Edit .env if needed (default: mongodb://localhost:27017/shopez)
npm run seed      # Seeds 16 products + admin + demo user
npm run dev       # Starts server on http://localhost:5000
```

2. Setup Client (new terminal)

```
cd client
npm install
npm run dev      # Starts React app on http://localhost:5173
```

5. Folder Structure

Client

client

```
|
|
|— public
|
|— src
|   |— components
|   |— pages
|   |— context
|   |— services
|   |— hooks
|   |— assets
|   |— App.jsx
|   └— main.jsx
```

Server

server

```
|  
|  
├── controllers  
├── middleware  
├── models  
├── routes  
├── config  
├── utils  
├── server.js  
└── package.json
```

```
ShopEZ/  
├── client/                # React frontend (Vite)  
│   └── src/  
│       ├── components/   # Navbar, Footer, ProductCard, etc.  
│       ├── context/      # AuthContext, CartContext  
│       ├── pages/        # All page components  
│       │   └── admin/    # Admin dashboard pages  
│       └── utils/        # Axios API instance  
└── server/               # Express backend  
    ├── config/           # MongoDB connection  
    ├── controllers/      # Business logic  
    ├── middleware/       # JWT auth  
    ├── models/           # Mongoose models  
    ├── routes/           # API routes  
    └── seed.js           # Database seeder
```

6. Running the Application

Backend

cd server

npm install

npm start

Backend runs at:

http://localhost:5000

Frontend

cd client

npm install

npm run dev

Frontend runs at:

http://localhost:5173

7. API Documentation

Authentication APIs

Method	Endpoint	Description
POST	/api/auth/register	Register User
POST	/api/auth/login	Login User

Product APIs

Method Endpoint

GET /api/products

GET /api/products/

POST /api/products

PUT /api/products/

DELETE /api/products/

Cart APIs

Method Endpoint

GET /api/cart

POST /api/cart/add

PUT /api/cart/update/

DELETE /api/cart/remove/

Order APIs

Method Endpoint

POST /api/orders

GET /api/orders

GET /api/orders/

PUT /api/orders/status/

User APIs

Method Endpoint

GET /api/users/profile

PUT /api/users/profile

 API Endpoints			
Method	Endpoint	Description	Auth
POST	/api/users/register	Register user	Public
POST	/api/users/login	Login	Public
GET	/api/products	Get all products	Public
GET	/api/products/:id	Get single product	Public
POST	/api/cart	Add to cart	User
POST	/api/orders	Create order	User
GET	/api/orders/my	My orders	User
GET	/api/admin/stats	Dashboard stats	Admin
GET	/api/admin/orders	All orders	Admin
PUT	/api/admin/orders/:id	Update order status	Admin

8. Authentication

ShopEZ uses JWT (JSON Web Token) authentication.

Authentication flow:

1. User registers.
2. Password is encrypted using bcrypt.
3. User logs in.
4. Server generates JWT Token.

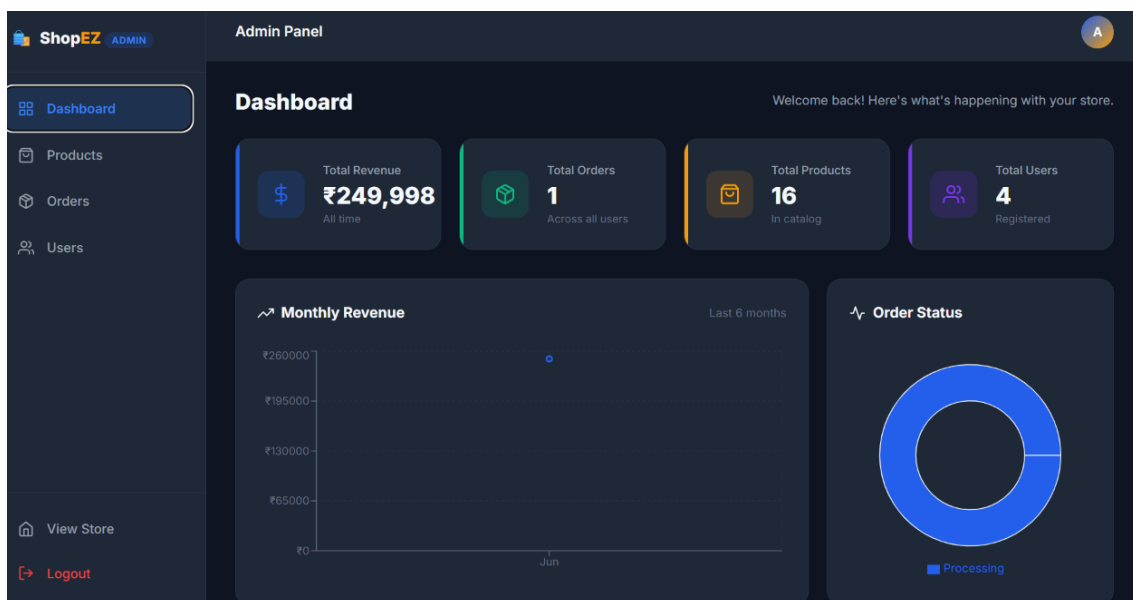
5. Token is stored on the client.
6. Protected routes validate the JWT before granting access.

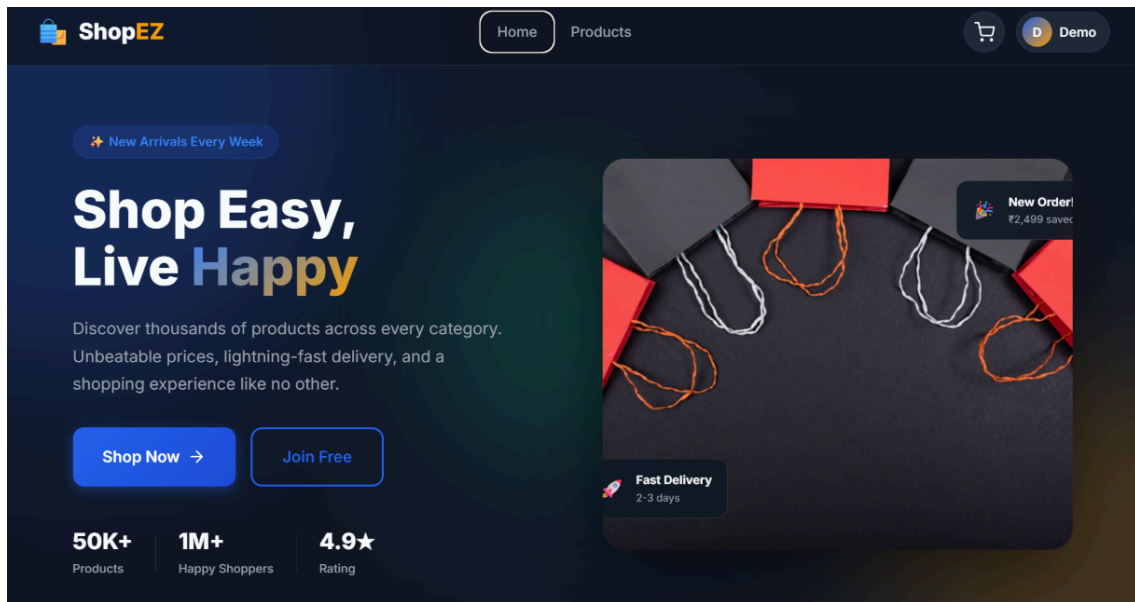
Role-based authorization separates Customer and Admin privileges.

9. User Interface

The application consists of responsive pages:

- Home Page
- Login Page
- Registration Page
- Products Page
- Product Details Page
- Shopping Cart
- Checkout
- Orders
- User Profile
- Admin Dashboard





10. Testing

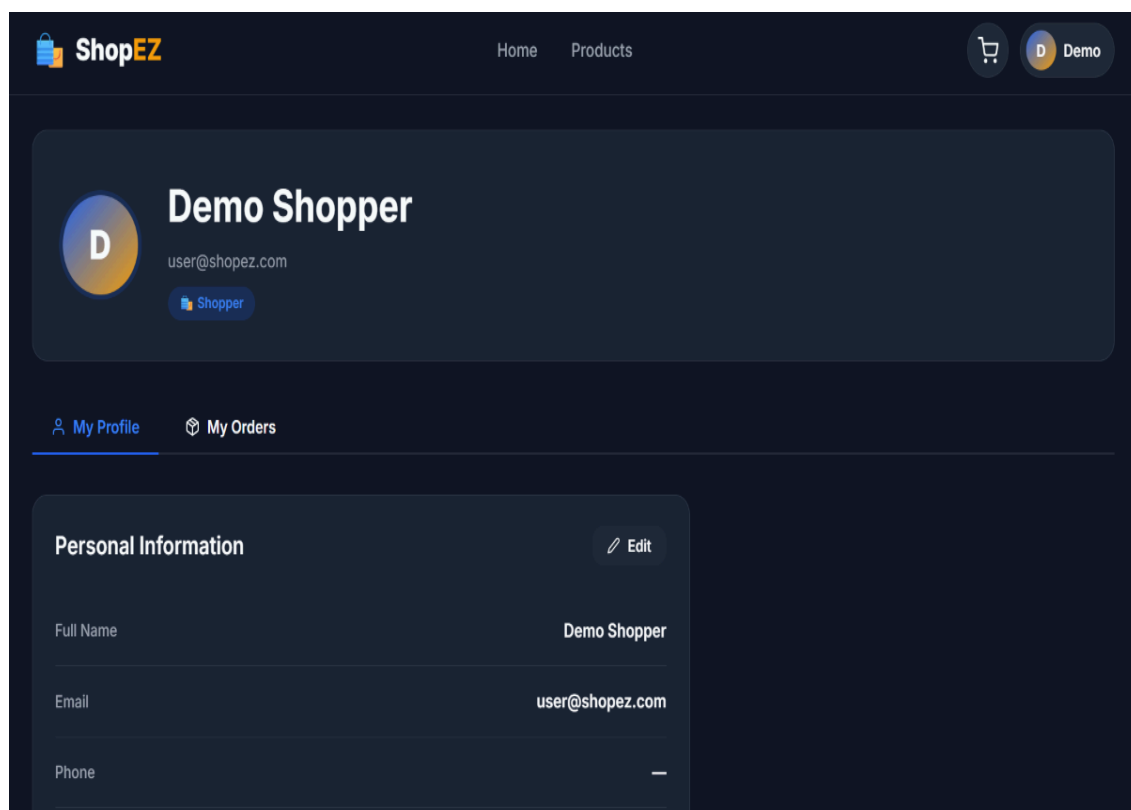
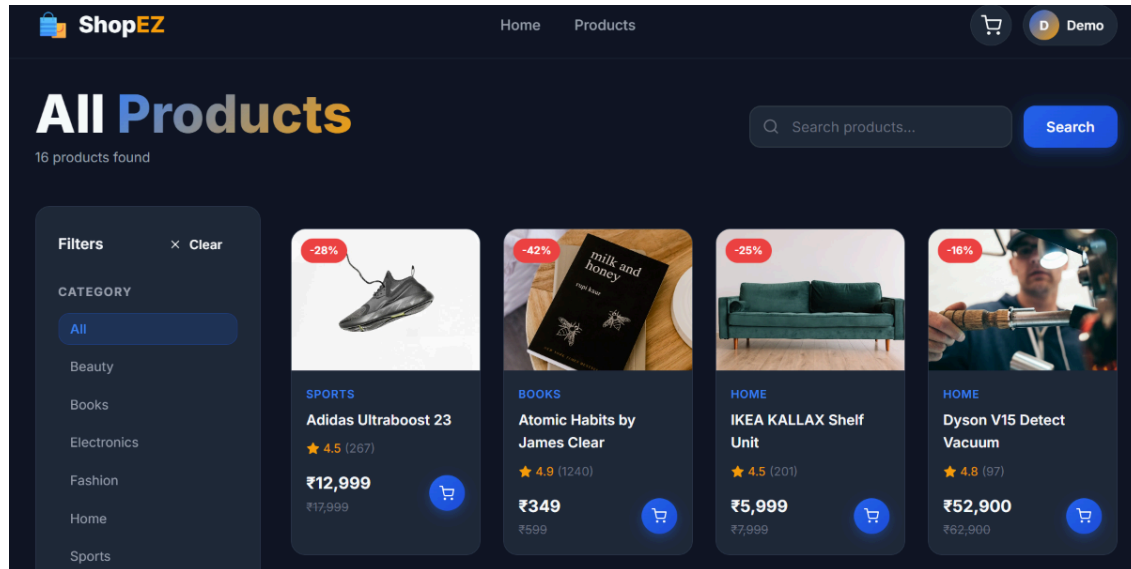
Testing included:

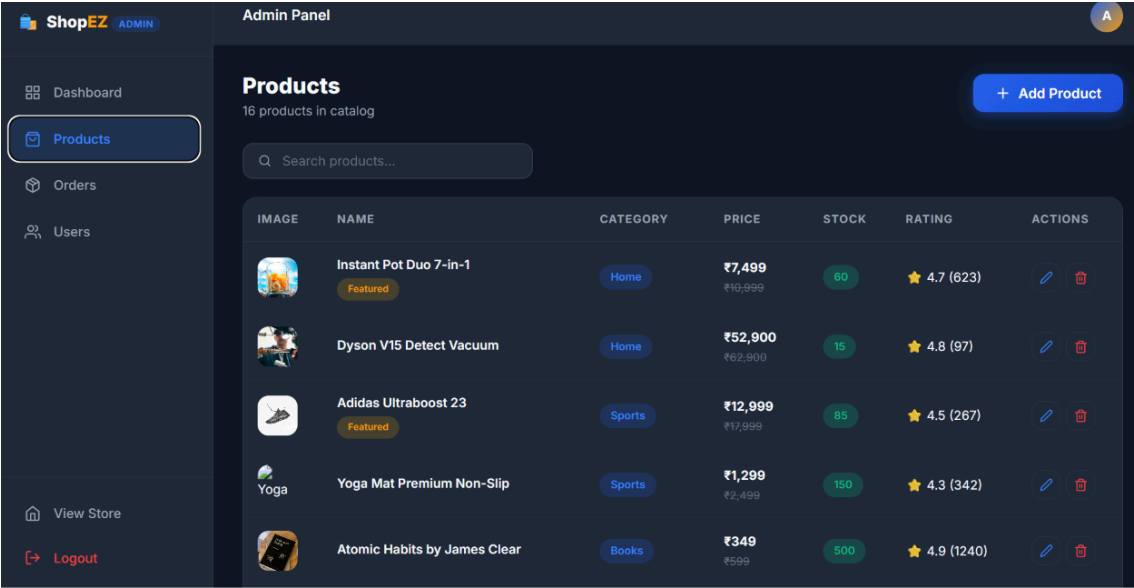
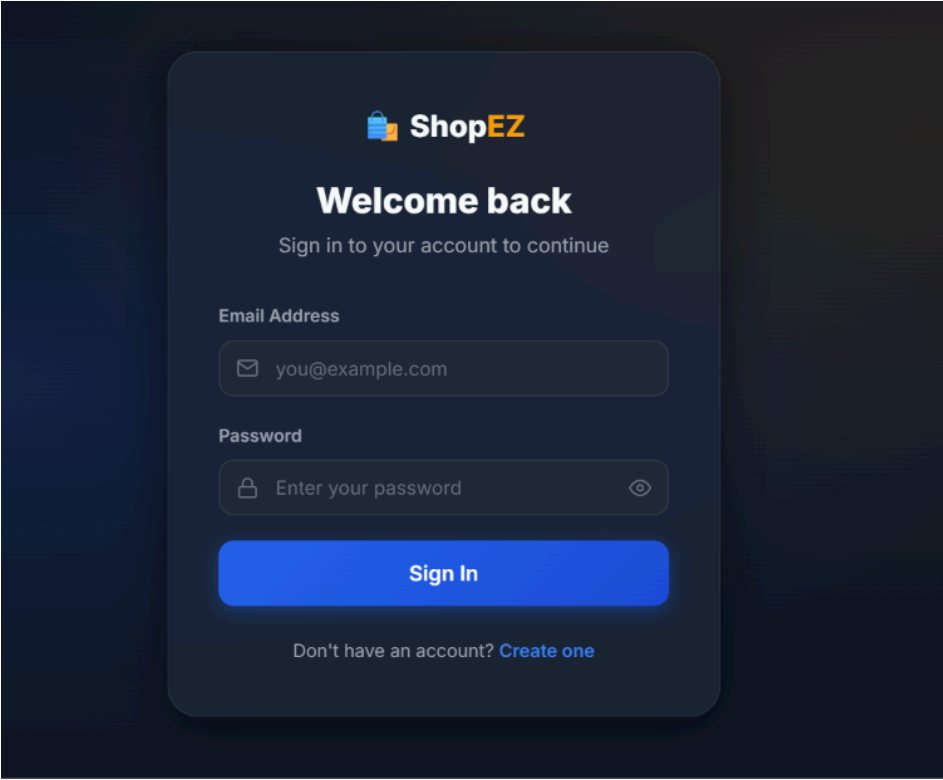
- Registration Testing
- Login Testing
- Product API Testing
- Cart Testing
- Checkout Testing
- Admin Testing
- JWT Authentication Testing
- MongoDB CRUD Testing

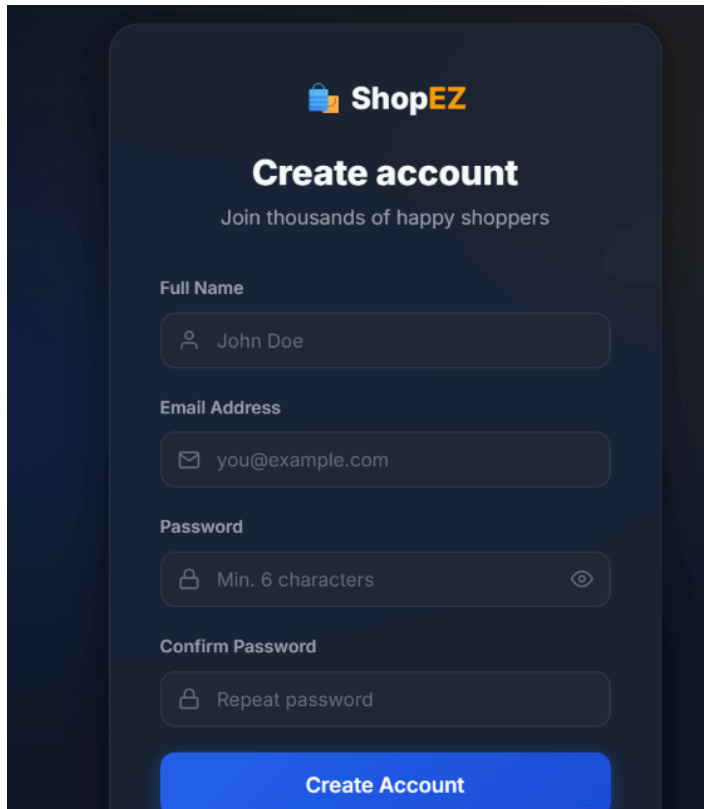
Tools Used:

- Postman
- Chrome Developer Tools
- MongoDB Compass

11. Screenshots or Demo





A dark-themed mobile app interface for creating an account. At the top is the ShopEZ logo with a shopping bag icon. Below it is the title 'Create account' and a subtitle 'Join thousands of happy shoppers'. The form consists of four input fields: 'Full Name' with a person icon and the text 'John Doe', 'Email Address' with an envelope icon and the text 'you@example.com', 'Password' with a lock icon, the text 'Min. 6 characters', and an eye icon for toggling visibility, and 'Confirm Password' with a lock icon and the text 'Repeat password'. At the bottom is a blue button labeled 'Create Account'.

12. Known Issues

- Payment Gateway is currently in demo mode.
 - Email verification is not implemented.
 - Wishlist feature is not available.
 - Product recommendations are basic.
 - Limited analytics in the admin dashboard.
-

13. Future Enhancements

Future improvements include:

- AI-based Product Recommendation System
- Wishlist Module
- Product Review & Rating System
- Multiple Payment Gateways

- Coupon & Discount Management
- Inventory Management
- Mobile Application (React Native)
- Push Notifications
- Chat Support
- Live Order Tracking
- Cloud Deployment using AWS or Azure
- Docker & Kubernetes Deployment
- Elasticsearch for Faster Search
- Multi-language Support
- Dark Mode
- Analytics Dashboard with Charts
- Microservices Architecture for Scalability